

Relatório de Auditoria de Segurança

Avaliação Técnica Sistemática de Riscos, Permissões e Integridade de Código

Data da Auditoria:	05 de Setembro de 2026
Escopo Auditado:	Repositório millersantosbr/Vou-de-Van-Alagoas (Branch: main)
Stack Detectada:	Next.js 16.1 (App Router, SSG Export), React 19, Cloud Firestore (Firebase SDK 11.10), Firebase Hosting, Tailwind CSS, Leaflet Maps.
Responsável Técnico:	Antigravity AI Security Suite (Engenharia de Segurança)
Status de Conclusão:	Concluída — 6 Achados Catalogados com Remediações

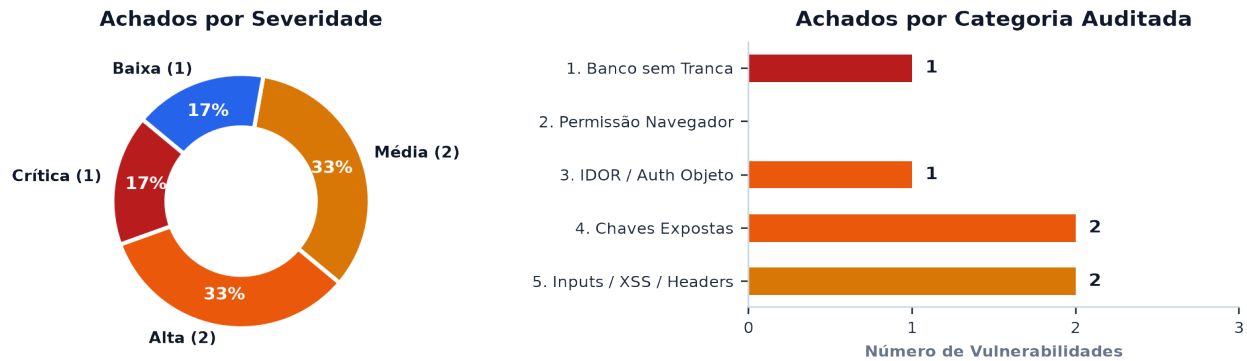
Nota Metodológica & Mapeamento de Stack

A auditoria cobriu rigorosamente cinco categorias essenciais de segurança adaptadas à arquitetura do projeto (aplicação web estática Next.js exportada em SSG vinculada a projeto Cloud Firestore no Firebase Hosting):

- BANCO SEM TRANCA (Isolamento de inquilino/dono):** Mapeado para as regras declarativas do Cloud Firestore (firestore.rules), investigando permissões globais de leitura e escritas sem validação de autorização de dono ou inquilino.
- PERMISSÃO DEFINIDA NO NAVEGADOR:** Mapeado para checagens de gates de papel (isAdmin/canEdit) no client Next.js vs validações correspondentes nas regras de banco.
- IDOR (Insecure Direct Object References):** Mapeado para manipulação direta de IDs em rotas Next.js (routes/[id]) e mutação de documentos Firestore por ID sem checagem de propriedade.
- CHAVES EXPOSTAS (Hardcode):** Varredura em código-fonte, arquivos raiz, configurações, variáveis de ambiente, histórico git e scripts de apoio.
- INPUTS SEM TRATAMENTO (XSS / Injeção):** Análise de dangerouslySetInnerHTML, sanitização de parâmetros de busca, injeção em URLs externas e cabeçalhos HTTP no firebase.json.

1. Resumo Executivo

A análise estática e estrutural identificou que a camada de frontend estático do Vou de Van possui boa higiene no consumo de dados imutáveis de itinerários, não apresentando pontos clássicos de XSS por interpolação de input de usuário. Entretanto, **duas vulnerabilidades críticas/altas** foram identificadas na infraestrutura do banco de dados (Cloud Firestore) e na gestão de segredos locais (chave de API de IA em texto puro), além da ausência de cabeçalhos de proteção no servidor de hosting.



CRÍTICA 1	ALTA 2	MÉDIA 2	BAIXA 1	PONTOS FORTES 4
--------------	-----------	------------	------------	--------------------

2. Pontos Fortes e Riscos Centrais

PONTOS FORTES (Auditado e Protegido)	PONTOS FRACOS (Riscos Centrais)
• Imutabilidade do Dataset: Os horários e rotas são consultados de um dataset estático imutável (lib/bus-data.ts), impedindo corrupção via banco. • Sanitização de URLs Externas: Parâmetros de GPS para Apple Maps e Google Maps usam encodeURIComponent e números estritos. • Higiene no Histórico Git: Não foram encontradas credenciais administrativas de deploy do Firebase nos commits públicos. • Isolamento no Client: Ausência de chamadas perigosas a eval() ou injeção de strings de busca em HTML bruto.	• Banco Globalmente Aberto: firestore.rules libera leitura completa para qualquer um e escrita para qualquer usuário logado. • Chave de IA em Texto Puro: Arquivo apikeyuilora.txt presente na raiz do repositório local. • Sem Cabeçalhos de Segurança HTTP: firebase.json não aplica CSP, X-Frame-Options nem mitigação de clickjacking. • Validação Silenciosa no Next.js: next.config.mjs configurado com ignoreBuildErrors: true.

3. Tabela de Achados Detalhados

Relação arquivo por arquivo, linha por linha de todas as vulnerabilidades detectadas:

CRÍTICA	1. Banco sem Tranca	firestore.rules:5-8	Regra coringa (/{document=**}) permitindo leitura pública irrestrita e escrita/exclusão global para qualquer usuário autenticado sem segregação de inquilino.
ALTA	3. IDOR / Auth	firestore.rules:7	Ausência de validação de propriedade (request.auth.uid == resource.data.ownerId) permitindo alteração e exclusão de objetos de terceiros via ID direto.
ALTA	4. Chaves Expostas	apikeyuilora.txt:1	Chave de API secreta de IA (sk-uilora-...) armazenada em texto puro na raiz do projeto, sujeita a vazamento e sequestro de cota.
MÉDIA	5. Headers / XSS	firebase.json:10-54	Ausência de cabeçalhos de segurança HTTP (CSP, X-Frame-Options contra Clickjacking, X-Content-Type-Options e Referrer-Policy).
MÉDIA	4. Chaves Expostas	lib/firebase.ts:5,10	Defaults inseguros com fallback permissivo sem validação de inicialização (startup fail-fast) para credenciais do Firebase SDK.
BAIXA	Higiene de Build	next.config.mjs:6	Parâmetro ignoreBuildErrors: true suprime checagens do compilador TypeScript, permitindo deploy com regressões e tipos inválidos.
INFO	2. Permissão Nav.	app/page.tsx	Não aplicável no frontend atual (aplicação de consulta pública sem painel administrativo ou gestão de usuários no client).

4. Recomendações Priorizadas

Prioridade	Ação Recomendada	Impacto na Segurança
P1 (Crítica)	Fechar regras do Firestore: Substituir match /{document=**} por regras estritas e fechadas por padrão (allow read, write: if false;) caso o Firestore não seja consumido no client, ou especificar coleções públicas de somente-leitura.	Elimina o risco de dump público de dados e remoção/destruição maliciosa do banco por qualquer usuário autenticado.
P1 (Alta)	Rotacionar e expurgar chaves em arquivo: Remover apikeyuilora.txt da raiz do projeto, invalidar a chave no console do provedor e adotar injeção exclusiva via variáveis de ambiente.	Previne sequestro de cotas pagas e uso não autorizado da API de IA.
P2 (Média)	Adicionar cabeçalhos HTTP no Firebase: Configurar no firebase.json os headers X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy e CSP estrita.	Protege usuários contra Clickjacking, ataques de MIME-sniffing e execução de scripts não autorizados.
P2 (Média)	Startup Validation de Env Vars: Em lib/firebase.ts, remover fallbacks de string vazia/dummy e validar variáveis com Zod no build.	Garante que a aplicação falhe imediatamente no build caso credenciais obrigatórias estejam ausentes.
P3 (Baixa)	Ativar verificação rigorosa de tipos: Remover ignoreBuildErrors: true em next.config.mjs.	Evita que quebras de código ou inconsistências estruturais passem despercebidas para produção.

5. Issues para o GitHub (Prontas para Copiar e Colar)

Abaixo constam os textos completos em Markdown de cada issue gerada a partir dos achados auditados. Cada bloco está delimitado para facilitar a inclusão direta no GitHub Issues.

--- ISSUE 1 ---

Título: [Segurança] Regras permissivas no Cloud Firestore expõem banco a leitura e mutação arbitrária

Labels: security, critical, database

Descrição do Problema:

O arquivo firestore.rules utiliza uma regra coringa match /{document=**} com allow read: if true; e allow write: if request.auth != null;. Isso expõe qualquer coleção presente ou futura para leitura irrestrita e permite a qualquer usuário autenticado criar, modificar ou apagar todos os documentos do banco sem segregação de inquilino ou validação de dono.

Evidência: Arquivo: firestore.rules:5-8

```
service cloud.firestore {  
  match /databases/{database}/documents {  
    match /{document=**} {  
      allow read: if true;  
      allow write: if request.auth != null;  
    }  
  }  
}
```

Impacto: Risco de destruição integral (wipe) de dados, alteração de tabelas e exposição de dados sensíveis.

Sugestão de Correção: Restringir as regras às coleções estritamente necessárias e proibir escrita global por padrão (allow write: if false;).

Critérios de Aceite:

- ☐ A regra recursiva aberta /{document=**} foi removida.
- ☐ Escrita global não autenticada/não-admin foi bloqueada.
- ☐ Testes de regras do Firestore executados com sucesso no emulador.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Chave de API de IA em texto puro na raiz do projeto (apikeyuilora.txt)

Labels: security, high, secrets

Descrição do Problema:

O arquivo apikeyuilora.txt contém uma credencial privada ativa de serviço de inteligência artificial (sk-uilora-...) armazenada em texto puro no sistema de arquivos do projeto. Embora o nome do arquivo tenha sido inserido no .gitignore, sua presença física no disco representa risco de vazamento em compartilhamentos, backups desprotegidos ou ambientes de CI/CD.

Evidência: Arquivo: apikeyuilora.txt:1 — sk-uilora-92yyHJiDHAP-XoikguP4ovWfcMWP7AIXWbBKjsqNHd4

Impacto: Consumo não autorizado de cota paga, acesso a recursos de terceiros e risco financeiro.

Sugestão de Correção: Revogar imediatamente a chave no painel do provedor, excluir o arquivo do disco e configurar consumo via variável de ambiente UI_LORA_API_KEY.

Critérios de Aceite:

- ☐ A chave exposta foi revogada e uma nova gerada.
- ☐ O arquivo apikeyuilora.txt foi excluído do repositório.
- ☐ Confirmado que a chave não está presente em commits do histórico git.

--- FIM ISSUE 2 ---

5. Issues para o GitHub (Continuação)

--- ISSUE 3 ---

Título: [Segurança] Ausência de cabeçalhos de segurança HTTP e CSP no Firebase Hosting

Labels: security, medium, headers, hosting

Descrição do Problema:

O arquivo firebase.json define parâmetros de Cache-Control, mas não configura cabeçalhos defensivos essenciais recomendados pela OWASP, como Content-Security-Policy, X-Frame-Options, X-Content-Type-Options e Referrer-Policy. Isso deixa a aplicação vulnerável a ataques de Clickjacking (incorporação em <iframe> malicioso) e MIME-confusion.

Evidência: Arquivo: firebase.json:10-54 (Bloco headers sem políticas de proteção de framing e MIME).

Impacto: Risco de sequestro de interface (Clickjacking) e execução indevida de scripts caso uma injeção seja explorada.

Sugestão de Correção: Inserir no bloco global headers do firebase.json:

- X-Frame-Options: DENY
- X-Content-Type-Options: nosniff
- Referrer-Policy: strict-origin-when-cross-origin
- Permissions-Policy: geolocation=(self), microphone=(), camera=()

Critérios de Aceite:

[] Cabeçalhos de segurança adicionados ao firebase.json.

[] Deploy executado e validação via curl -I https://voudevan-al.web.app confirmando a presença dos headers.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Defaults inseguros e falta de validação de startup em variáveis de ambiente

Labels: security, medium, configuration

Descrição do Problema:

Em lib/firebase.ts, variáveis de ambiente críticas utilizam operadores de fallback silencioso (process.env.NEXT_PUBLIC_FIREBASE_API_KEY || "AlzaSyDummyKeyForVouDeVanAlagoas"). Se as variáveis não forem supridas no build, a aplicação inicializa com credenciais inválidas silenciosamente, sem lançar erro (fail-fast). Além disso, em next.config.mjs, o parâmetro ignoreBuildErrors: true suprime falhas de compilação.

Evidência: Arquivos: lib/firebase.ts:5,10 e next.config.mjs:6

Impacto: Falhas silenciosas em produção e mascaramento de erros críticos de tipagem e ambiente.

Sugestão de Correção: Criar módulo de validação de ambiente com zod e desativar ignoreBuildErrors.

Critérios de Aceite:

[] Validação com Zod implementada garantindo fail-fast na ausência de variáveis obrigatórias.

[] ignoreBuildErrors: true removido de next.config.mjs com compilação limpa do TypeScript.

--- FIM ISSUE 4 ---